# Unit III: Design of Sequential Logic

Flip-Flop: SR, J-K, D, T; Preset &Clear, Master-Slave JK Flip Flops, Truth Tables and Excitation tables, Conversion from one type to another type of Flop- Flop. Registers: SISO, SIPO, PISO, PIPO, Shift Registers, Bidirectional Shift Registers, **Counters**: Asynchronous Counter, Synchronous Counter, BCD Counter, Johnson Counter, Modulus of the counter (IC 7490), Sequence Generators.

Sequential logic circuit consists of a combinational circuit with storage elements connected as a feedback to combinational circuit

- output depends on the sequence of inputs (past and present)
- stores information (state) from past inputs

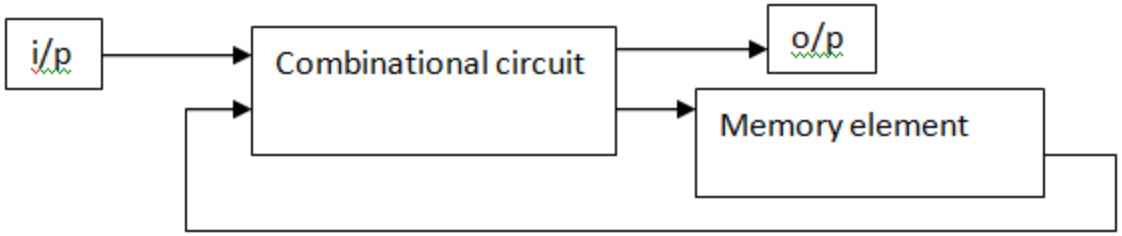
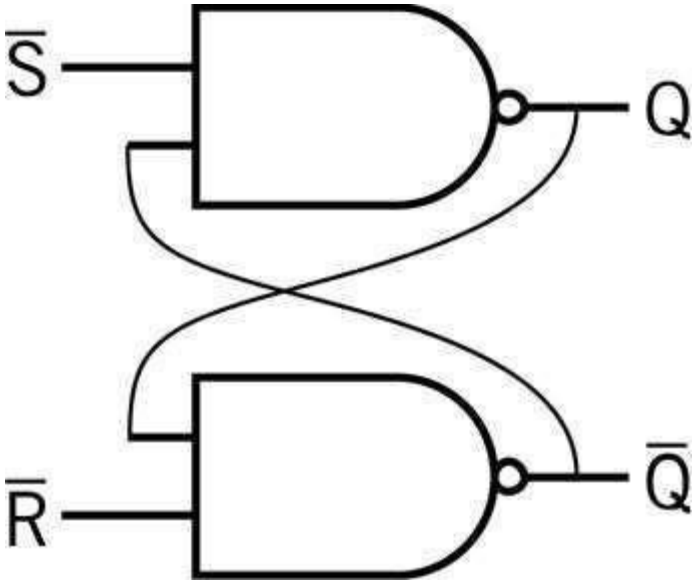



Figure 1: Sequential logic circuits

# SEQUENTIAL LOGIC CIRCUITS

- Output depends on
  - Input
  - Previous state of the circuit
- Flip-flop: basic memory element
- State table: output for all combinations of input and previous states(Truth Table)



# SEQUENTIAL LOGIC CIRCUITS

1. Sequential circuit receives the binary information from external inputs and with the present state of the storage elements together determine the binary value of the outputs.
2. The output in a sequential circuit are a function of not only the inputs, but also the present state of the storage elements.
3. The next state of the storage elements is also a function of external inputs and the present state.
4. There are two main types of sequential circuits
   1. synchronous sequential circuits
   2. asynchronous sequential circuits

# SEQUENTIAL LOGIC CIRCUITS

## Synchronous sequential circuits

It is a system whose behaviour can be defined from the knowledge of its signals at discrete instants of time.

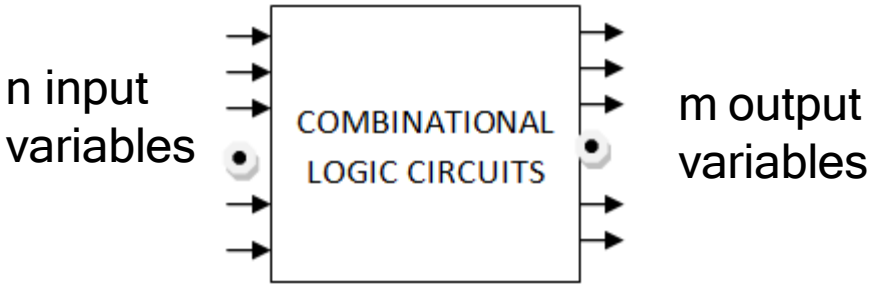
## Asynchronous sequential circuits

It depends upon the input signals at any instant of time and the order in which the input changes.

# Combinational vs. Sequential

## COMBINATIONAL LOGIC CIRCUIT

Combinational logic circuit consists of input variables, logic gates and output variables. The logic gate accepts signals from the inputs and generates signals to the outputs.




For n input variables there are 2n possible combinations of binary input variables

# SEQUENTIAL LOGIC CIRCUITS

Sequential logic circuit consists of a combinational circuit with storage elements connected as a feedback to combinational circuit.

- output depends on the sequence of inputs (past and present)
- stores information (state) from past inputs

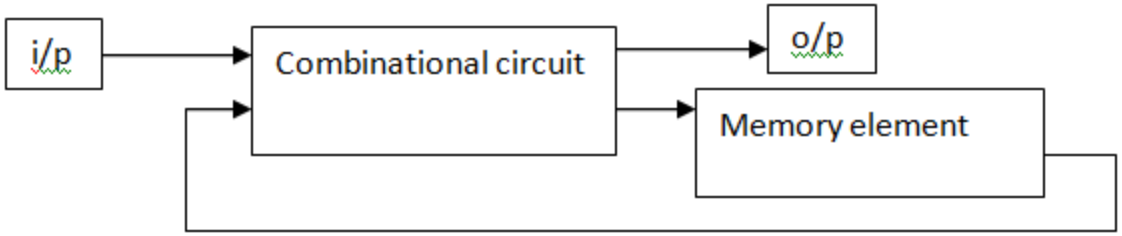



Figure 1: Sequential logic circuits

# Combinational vs. Sequential

## Combinational Circuit

always gives the same output for a given set of inputs

ex: adder always generates sum and carry,
regardless of previous inputs

## Sequential Circuit

stores information

output depends on stored information (state) plus input

so a given input might produce different outputs,
depending on the stored information

*example:* ticket counter

# Combinational vs. Sequential

| Combinational Logic Circuits | Sequential Logic Circuits |
|---|---|
| Output is a function of the present inputs (Time Independent Logic). | Output is a function of clock, present inputs and the previous states of the system. |
| Do not have the ability to store data (state). | Have memory to store the present states that is sent as control input (enable) for the next operation. |
| It does not require any feedback. It simply outputs the input according to the logic designed. | It involves feedback from output to input that is stored in the memory for the next operation. |
| Used mainly for Arithmetic and Boolean operations. | Used for storing data (and hence used in RAM). |
| Logic gates are the elementary building blocks. | Flip flops (binary storage device) are the elementary building unit. |
| Independent of clock and hence does not require triggering to operate. | Clocked (Triggered for operation with electronic pulses). |
| Example: Adder [1+0=1; Dependency only on present inputs i.e., 1 and 0]. | Example: Counter [Previous O/P +1=Current O/P; Dependency on present input as well as previous state]. |

# LATCHES

## STORAGE ELEMENTS

Storage elements in a digital circuit can maintain a binary state indefinitely, until directed by an input signal to switch states. The major difference among various storage elements are the number of input they posses and the manner in which the inputs affect the binary state.There are two types of storage elements

1. Latches
2. Flipflops

Storage elements that operate with signal level are referred as latch and those controlled by a clock transition are referred as flipflops.

# LATCHES

## 1. LATCHES:

A latch has a feedback path, so information can be retained by the device. Therefore latches can be memory devices, and can store one bit of data for as long as the device is powered. As the name suggests, latches are used to "latch onto" information and hold in place. Latches are very similar to flip-flops, but are not synchronous devices, and do not operate on clock edges as flip-flops do. Latch is a level sensitive device. Latch is a monostable multivibrator.

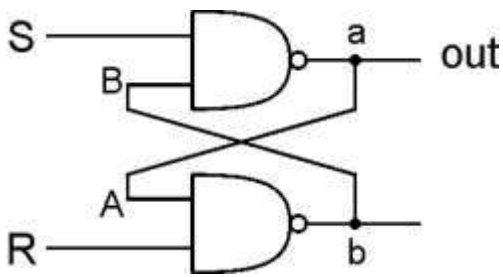
## 2. FLIPFLOPS:

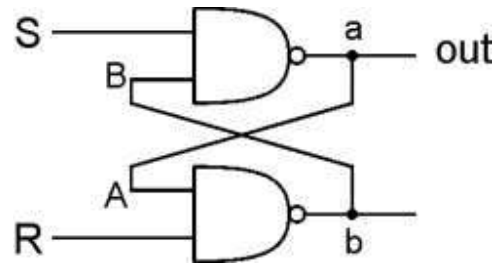
A **flip-flop** is a circuit that has two stable states and can be used to store state information. A flip-flop is a bistable multivibrator. The circuit can be made to change state by signals applied to one or more control inputs and will have one or two outputs. It is the basic storage element in sequential logic. Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems. Flipflop is a edge sensitive device.

# LATCHES

## SR LATCH

An **SR latch** (Set/Reset) is an asynchronous device: it works independently of control signals and relies only on the state of the S and R inputs. In the image we can see that an SR flip-flop can be created with two NOR gates that have a cross-feedback loop. SR latches can also be made from NAND gates, but the inputs are swapped and negated. In this case, it is sometimes called an **SR latch**.




R is used to ―reset‖ or ―clear‖ the element – set it to zero. S is used to ―set‖ the element – set it to one.

If both R and S are one, out could be <u>either</u> zero or one. ―quiescent‖ state -- holds its previous value. note: if a is 1, b is 0, and vice versa

# LATCHES

## GATED D-LATCH

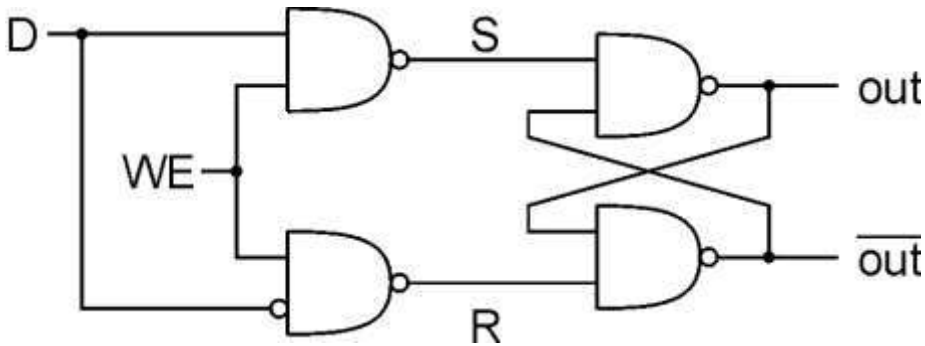
The **D latch** (D for "data") or **transparent latch** is a simple extension of the gated SR latch that removes the possibility of invalid input states. Two inputs: D (data) and WE (write enable)

when WE = 1, latch is set to value of D

S = NOT(D), R = D

when WE = 0, latch holds previous value

S = R = 1



# FLIPFLOPS:RS FLIPFLOP

## Flip flops

A flip flop is an electronic circuit with two stable states that can be used to store binary data. The stored data can be changed by applying varying inputs. Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems. Flip-flops and latches are used as data storage elements. There are 4 types of flipflops

1. RS flip flop
2. Jk flip flop
3. D flip flop
4. T flip flop

Applications of Flip-Flops

These are the various types of flip-flops being used in digital electronic circuits and the applications like Counters, Frequency Dividers, Shift Registers, Storage Registers

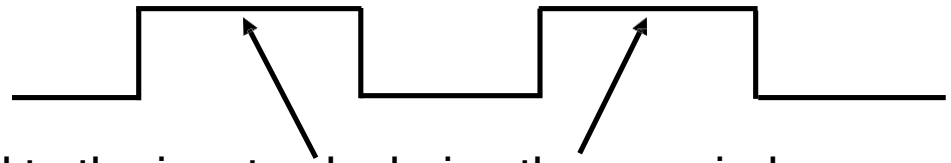
# FLIPFLOPS:RS FLIPFLOP

EDGE-TRIGGERED FLIP FLOPS

**Characteristics**

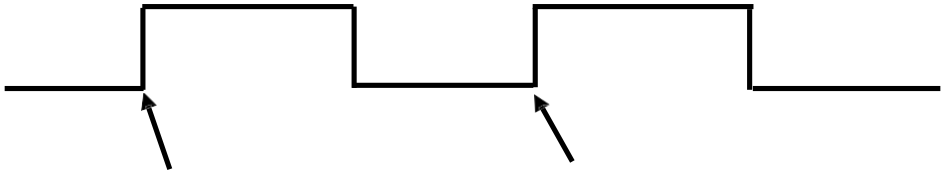
- **State transition occurs at the rising edge or falling edge of the clock pulse**

**Latches**



respond to the input only during these periods

**Edge-triggered Flip Flops (positive)**

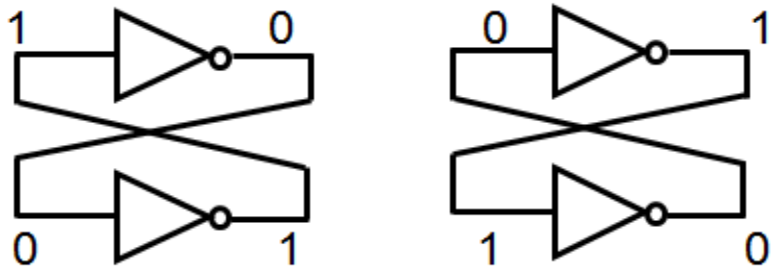


respond to the input only at this time

# FLIPFLOPS:RS FLIPFLOP

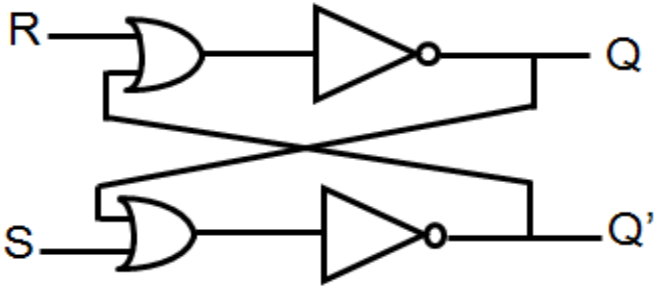
**Characteristics**

- **2 stable states**
- **Memory capability**
- **Operation is specified by a Characteristic Table**




**0-state** **1-state**

**In order to be used in the computer circuits, state of the flip flop should have input terminals and output terminals so that it can be set to a certain state, and its state can be read externally.**

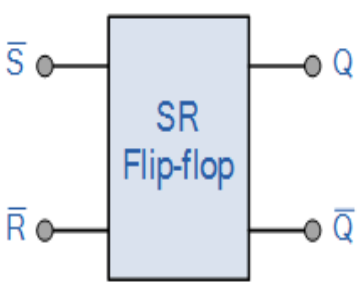



| S | R | Q(t+1) |
|---|---|---|
| 0 | 0 | Q(t) |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | indeterminate (forbidden) |

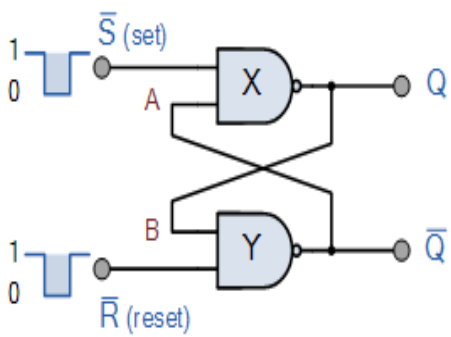
# FLIPFLOPS:SR FLIPFLOP

## SR Flip-Flop

The **SR flip-flop**, also known as a *SR Latch*, can be considered as one of the most basic sequential logic circuit possible. This simple flip-flop is basically a one-bit memory bistable device that has two inputs, one which will ―SET‖ the device (meaning the output = ―1‖), and is labelled **S** and one which will ―RESET‖ the device (meaning the output = ―0‖), labelled **R**.The reset input resets the flip-flop back to its original state with an output Q that will be either at a logic level ―1‖or logic ―0‖depending upon this set/reset condition.
A basic NAND gate SR flip-flop circuit provides feedback from both of its outputs back to its opposing inputs and is commonly used in memory circuits to store a single data bit.
Then the SR flip-flop actually has three inputs, Set, Reset and its current output Q relating to it's current state or history.




Symbol




Circuit

| State | S | R | Q | | Description |
|---|---|---|---|---|---|
| Set | 1 | 0 | 0 | 1 | Set $\overline{Q}$ » 1 |
| | 1 | 1 | 0 | 1 | no change |
| Reset | 0 | 1 | 1 | 0 | Reset $\overline{Q}$ » 0 |
| | 1 | 1 | 1 | 0 | no change |
| Invalid | 0 | 0 | 1 | 1 | Invalid Condition |

Truth Table for this Set-Reset Function

# FLIPFLOPS:JK FLIPFLOP

## JK Flip flop

The JK Flip-flop is similar to the SR Flip-flop but there is no change in state when the J and K inputs are both LOW. The basic S-R NAND flip-flop circuit has many advantages and uses in sequential logic circuits but it suffers from two basic switching problems.

1. the Set = 0 and Reset = 0 condition (S = R = 0) must always be avoided
2.if Set or Reset change state while the enable (EN) input is high the correct latching action may not occur

Then to overcome these two fundamental design problems with the SR flip-flop design, the **JK flip Flop** was developed by the scientist name Jack Kirby.

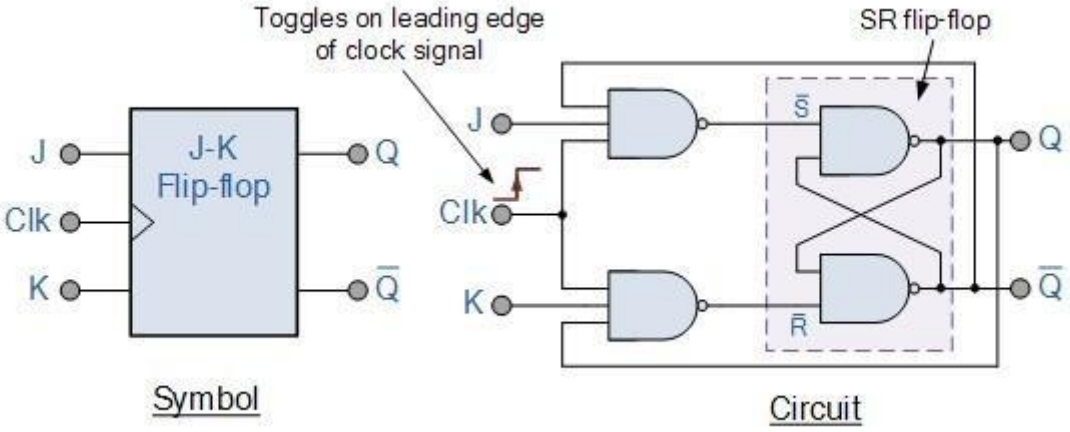
The **JK flip flop** is basically a gated SR flip-flop with the addition of a clock input circuitry that prevents the illegal or invalid output condition that can occur when both inputs S and R are equal to logic level ―1‖. Due to this additional clocked input, a JK flip-flop has four possible input combinations, ―logic 1‖, ―logic 0‖, ―no change‖ and ―toggle‖. The symbol for a JK flip flop is similar to that of an *SR Bistable Latch* as seen in the previous tutorial except for the addition of a clock input.

# FLIPFLOPS:JK FLIPFLOP

❖ Both the S and the R inputs of the previous SR bistable have now been replaced by two inputs called the J and K inputs, respectively after its inventor Jack Kilby. Then thisequates to: J = S and K = R.

❖ The two 2-input AND gates of the gated SR bistable have now been replaced by two 3-input NAND gates with the third input of each gate connected to the outputs at Q and Q. This cross coupling of the SR flip-flop allows the previously invalid condition of S = —1‖ and R = —1‖ state to be used to produce a —toggle action‖ as the two inputs are now interlocked.

❖ If the circuit is now —SET‖ the J input is inhibited by the —0‖ status of Q through the lower NAND gate. If the circuit is —RESET‖ the K input is inhibited by the —0‖ status of Q through the upper NAND gate. As Q and Q are always different we can use them to control the input. When both inputs J and K are equal to logic —1‖, the JK flip flop toggles as shown in the following truth table.



# FLIPFLOPS:JK FLIPFLOP

## The Truth Table for the JK Function

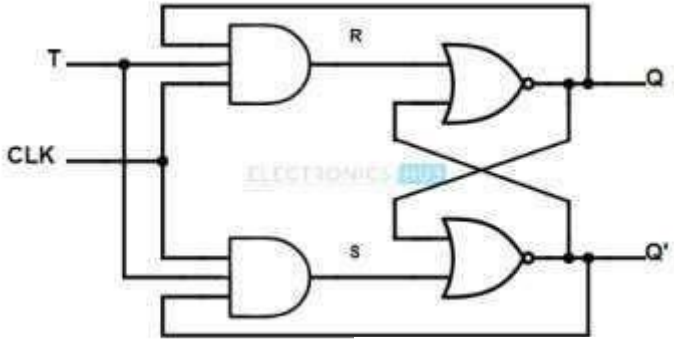
Then the JK flip-flop is basically an SR flip flop with feedback which enables only one of its two input terminals, either SET or RESET to be active at any one time thereby eliminating the invalid condition seen previously in the SR flip flop circuit. Also when both the J and the K inputs are at logic level ―1‖ at the same time, and the clock input is pulsed ―HIGH‖, the circuit will ―toggle‖ from its SET state to a RESET state, or visa-versa. This results in the JK flip flop acting more like a T-type toggle flip-flop when both terminals are―HIGH‖.

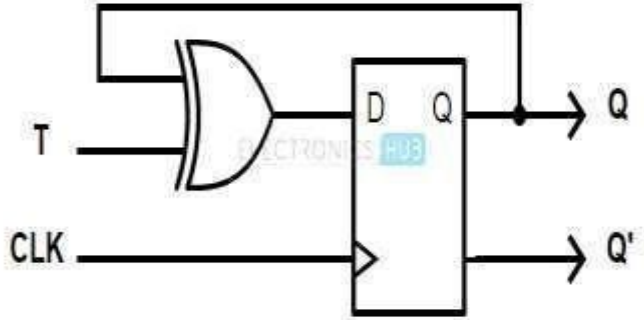
| | Input | | Output | | Description |
|---|---|---|---|---|---|
| | J | K | Q | $\overline{Q}$ | |
| same as for the SR Latch | 0 | 0 | 0 | 0 | Memory no change |
| | 0 | 0 | 0 | 1 | |
| | 0 | 1 | 1 | 0 | Reset Q » 0 |
| | 0 | 1 | 0 | 1 | |
| | 1 | 0 | 0 | 1 | Set Q » 1 |
| | 1 | 0 | 1 | 0 | |
| toggle action | 1 | 1 | 0 | 1 | Toggle |
| | 1 | 1 | 1 | 0 | |

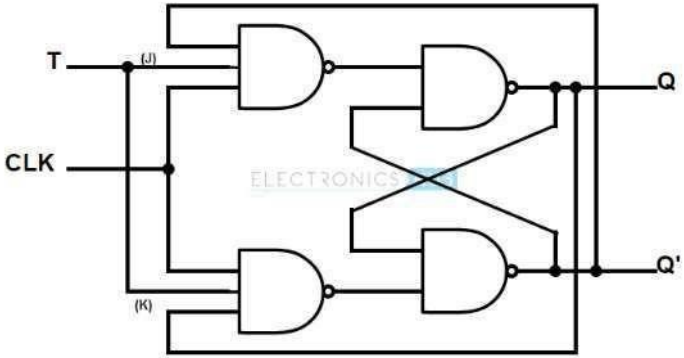
# FLIPFLOPS:T FLIPFLOP

## T FLIP FLOP

We can construct a T flip flop by any of the following methods. Connecting the output feedback to the input, in SR flip flop. Connecting the XOR of T input and Q PREVIOUS output to the Data input, in D flip flop. Hard – wiring the J and K inputs together and connecting it to T input, in JK flip – flop.



# FLIPFLOPS:T FLIPFLOP

## Working

T flip – flop is an edge triggered device i.e. the low to high or high to low transitions on a clock signal of narrow triggers that is provided as input will cause the change in output state of flip – flop. T flip – flop is an edge triggered device.

## Truth Table of T flip – flop

| | Previous | | Next | |
|---|---|---|---|---|
| T | $Q_{Prev}$ | $Q'_{Prev}$ | $Q_{Next}$ | $Q'_{Next}$ |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 |

# FLIPFLOPS:T FLIPFLOP

❖If the output Q = 0, then the upper NAND is in enable state and lower NAND gate is in disable condition. This allows the trigger to pass the S inputs to make the flip – flop in SET state i.e. Q = 1.

❖If the output Q = 1, then the upper NAND is in disable state and lower NAND gate is in enable condition. This allows the trigger to pass the R inputs to make the flip – flop in RESET state i.e. Q =0.

In simple terms, the operation of the T flip – flop is

When the T input is low, then the next sate of the T flip flop is same as the present state.

T = 0 and present state = 0 then the next state = 0

T = 1 and present state = 1 then the next state = 1

When the T input is high and during the positive transition of the clock signal, the next stateof the T flip – flop is the inverse of present state.

T = 1 and present state = 0 then the next state = 1

T = 1 and present state = 1 then the next state = 0

## Applications

Frequency Division Circuits.

2 – Bit Parallel Load Registers.

# FLIPFLOPS:D FLIPFLOP

## D FLIP FLOP

The D-type flip-flop is a modified Set-Reset flip-flop with the addition of an inverter to prevent the S and R inputs from being at the same logic level

One of the main disadvantages of the basic **SR NAND Gate Bistable** circuit is that the indeterminate input condition of SET = ―0‖ and RESET = ―0‖ is forbidden.

This state will force both outputs to be at logic ―1‖, over-riding the feedback latching action and whichever input goes to logic level ―1‖ first will lose control, while the other input still at logic ―0‖ controls the resulting state of the latch.

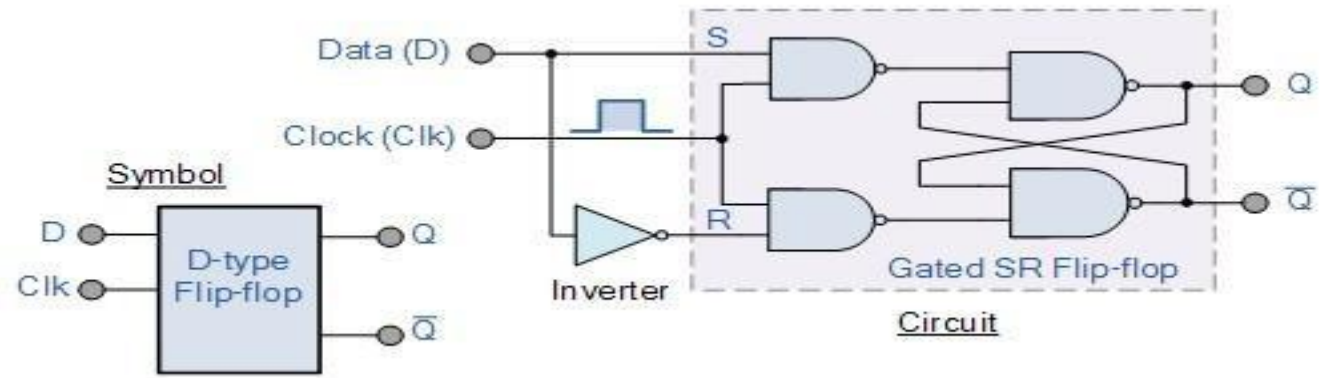
But in order to prevent this from happening an inverter can be connected between the ―SET‖ and the ―RESET‖ inputs to produce another type of flip flop circuit known as a **Data Latch**, **Delay flip flop**, **D-type Bistable**, **D-type Flip Flop** or just simply a **D Flip Flop** as it is more generally called.

The **D Flip Flop** is by far the most important of the clocked flip-flops as it ensures that ensures that inputs S and R are never equal to one at the same time. The D-type flip flop are constructed from a gated SR flip-flop with an inverter added between the S and the R inputs to allow for a single D (data) input.

Then this single data input, labelled ―D‖ and is used in place of the ―Set‖ signal, and the inverter is used to generate the complementary ―Reset‖ input thereby making a level-sensitive D-type flip-flop from a level-sensitive SR-latch as now S = D and R = not D as shown.

# FLIPFLOPS:D FLIPFLOP

## D-type Flip-Flop Circuit




We remember that a simple SR flip-flop requires two inputs, one to ―SET‖ the output and one to ―RESET‖ the output. By connecting an inverter (NOT gate) to the SR flip-flop we can ―SET‖ and ―RESET‖ the flip-flop using just one input as now the two input signals are complements of each other. This complement avoids the ambiguity inherent in the SR latch when both inputs are LOW, since that state is no longer possible. Thus this single input is called the ―DATA‖ input. If this data input is held HIGH the flip flop would be ―SET‖ and when it is LOW the flip flop would change and become ―RESET‖. However, this would be rather pointless since the output of the flip flop would always change on every pulse applied to this data input.

# FLIPFLOPS:D FLIPFLOP

To avoid this an additional input called the ―CLOCK‖ or ―ENABLE‖ input is used to isolate the data input from the flip flop's latching circuitry after the desired data has been stored. The effect is that D input condition is only copied to the output Q when the clock input is active. This then forms the basis of another sequential device called a **D Flip Flop**.

The ―Dflip flop‖ will store and output whatever logic level is applied to its data terminal so long as the clock input is HIGH. Once the clock input goes LOW the ―set‖ and ―reset‖ inputs of the flip-flop are both held at logic level ―1‖so it will not change state and store whatever data was present on its output before the clock transition occurred. In other words the output is ―latched‖at either logic ―0‖or logic ―1‖.

## Truth Table for the D-type Flip Flop

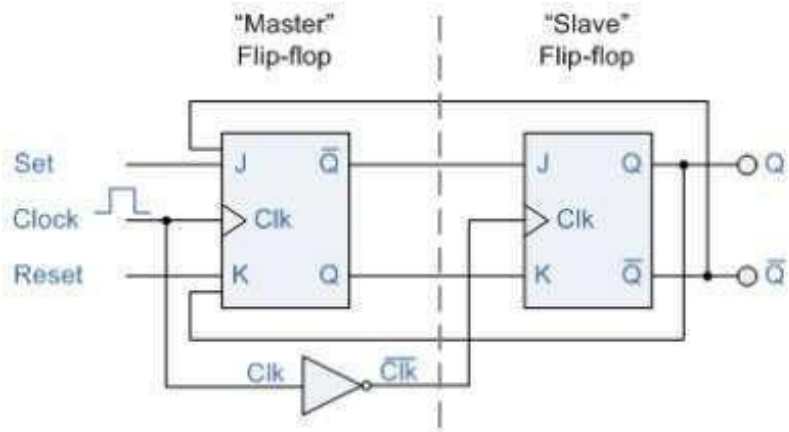
| Clk | D | Q | | Description |
|---|---|---|---|---|
| ↓ » 0 | X | Q | $\overline{Q}$ | Memory no change |
| ↑ » 1 | 0 | 0 | 1 | Reset Q » 0 |
| ↑ » 1 | 1 | 1 | 0 | Set Q » 1 |

Note that: ↓ and ↑ indicates direction of clock pulse as it is assumed D-type flip flops are edge triggered

# FLIPFLOPS:MASTER SLAVE FLIPFLOP

## MASTER SLAVE FLIPFLOP

Master-slave flip flop is designed using two separate flip flops. Out of these, one acts as the master and the other as a slave. The figure of a master-slave J-K flip flop is shown below.




Master Slave Flip Flop

From the above figure you can see that both the J-K flip flops are presented in a series connection. The output of the master J-K flip flop is fed to the input of the slave J-K flip flop. The output of the slave J-K flip flop is given as a feedback to the input of the master J-K flip flop. The clock pulse [Clk] is given to the master J-K flip flop and it is sent through a NOT Gate and thus inverted before passing it to the slave J-K flip flop.

# FLIPFLOPS:MASTER SLAVE FLIPFLOP

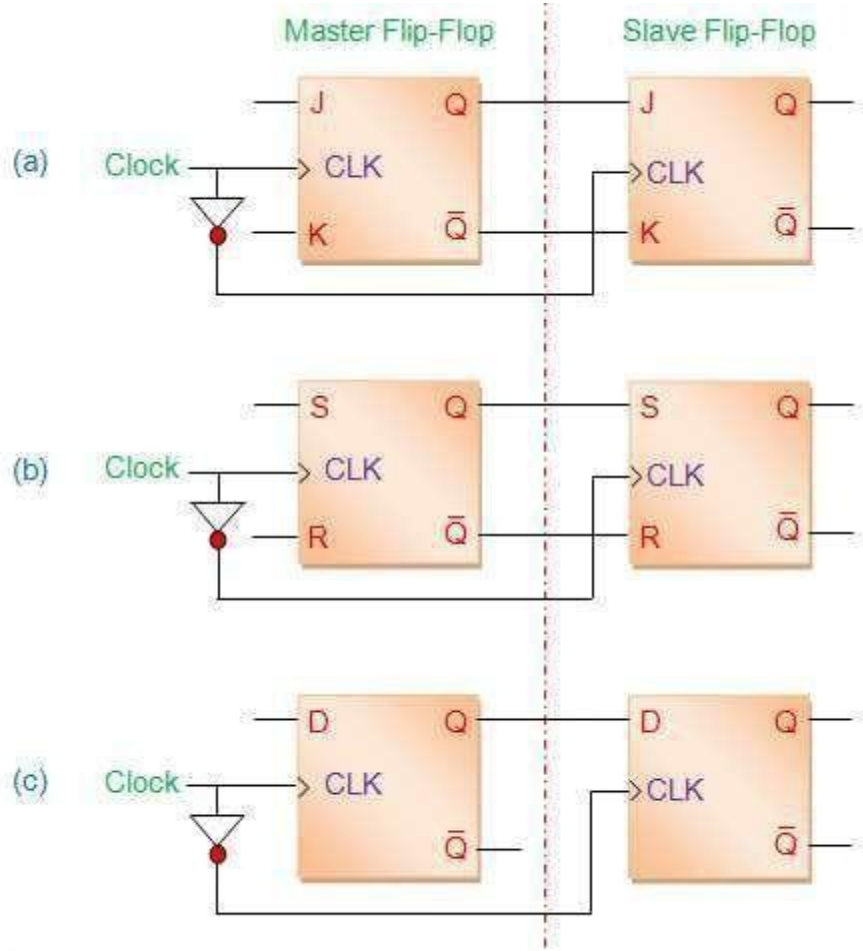



Figure 1 (a) Master-Slave JK flip-flop (b) Master-Slave SR flip-flop (c) Master-Slave D flip-flop

# FLIPFLOPS:MASTER SLAVE FLIPFLOP

The truth table corresponding to the working of the flip-flop shown in Figure is given by Table I. Here it is seen that the outputs at the master-part of the flip-flop (data enclosed in red boxes) appear during the positive-edge of the clock (red arrow). However at this instant the slave-outputs remain latched or unchanged. The same data is transferred to the output pins of the master-slave flip-flop (data enclosed in blue boxes) by the slave during the negative edge of the clock pulse (blue arrow). The same principle is further emphasized in the timing diagram of **master-slave flip-flop** shown by Figure 3. Here the green arrows are used to indicate that the slave-output is nothing but the master-output delayed by half-a-clock cycle. Moreover it is to be noted that the working of any other type of master-slave flip-flop is analogous to that of the master slave JK flip-flop explained here.

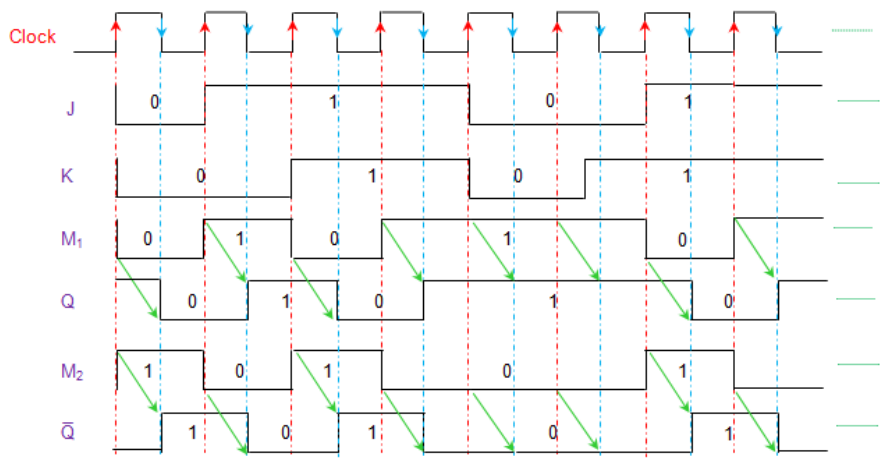



Figure 3 Timing diagram for master-slave JK flip-flop

# FLIPFLOPS:MASTER SLAVE FLIPFLOP

## Truth Table

| Trigger | Inputs | | Output | | | | | | Inference |
|---|---|---|---|---|---|---|---|---|---|
| | | | Present State | | Intermediate | | Next State | | |
| CLK | J | K | Q | $\bar{Q}$ | $M_1$ | $M_2$ | Q | $\bar{Q}$ | |
| ↑ | 0 | 0 | 0 | 1 | 0 | 1 | Latched | | No Change |
| ↓ | | | 0 | 1 | Latched | | 0 | 1 | |
| ↑ | | | 1 | 0 | 1 | 0 | Latched | | |
| ↓ | | | 1 | 0 | Latched | | 1 | 0 | |
| ↑ | 0 | 1 | 0 | 1 | 0 | 1 | Latched | | Reset |
| ↓ | | | 0 | 1 | Latched | | 0 | 1 | |
| ↑ | | | 1 | 0 | 0 | 1 | Latched | | |
| ↓ | | | 1 | 0 | Latched | | 0 | 1 | |
| ↑ | 1 | 0 | 0 | 1 | 1 | 0 | Latched | | Set |
| ↓ | | | 0 | 1 | Latched | | 1 | 0 | |
| ↑ | | | 1 | 0 | 1 | 0 | Latched | | |
| ↓ | | | 1 | 0 | Latched | | 1 | 0 | |
| ↑ | 1 | 1 | 0 | 1 | 1 | 0 | Latched | | Toggles |
| ↓ | | | 0 | 1 | Latched | | 1 | 0 | |
| ↑ | | | 1 | 0 | 0 | 1 | Latched | | |
| ↓ | | | 1 | 0 | Latched | | 0 | 1 | |

Table I Truth table for master-slave JK flip-flop

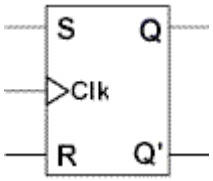
# FLIPFLOPS:EXCITATION FUNCTIONS

In electronics design, an **excitation table** shows the minimum inputs that are necessary to generate a particular next state (in other words, to **"excite"** it to the next state) when the current state is known. They are similar to truth tables and state tables, but rearrange the data so that the current state and next state are next to each other on the left-hand side of the table, and the inputs needed to make that state change happen.
All flip-flops can be divided into four basic types: **SR**, **JK**, **D** and **T**. They differ in the number of inputs and in the response invoked by different value of inputsignals.

The ***characteristic table*** in the third column of Table 1 defines the state of each flip-flop as a function of its inputs and previous state. **Q** refers to the present state and **Q(next)** refers to the next state after the occurrence of the clock pulse. The characteristic table for the RS flip-flop shows that the next state is equal to the present state when both inputs S and R are equal to 0. When R=1, the next clock pulse clears the flip-flop. When S=1, the flip-flop output Q is set to 1. The equation mark (?) for the next state when S and R are both equal to 1 designates an indeterminate next state.
The characteristic table for the JK flip-flop is the same as that of the RS when J and K are replaced by S and R respectively, except for the indeterminate case. When both J and K are equal to 1, the next state is equal to the complement of the present state, that is, Q(next) = Q'.
The next state of the D flip-flop is completely dependent on the input D and independent of the present state.
The next state for the T flip-flop is the same as the present state Q if T=0 and complemented if T=1.

# FLIPFLOPS:EXCITATION FUNCTIONS

## SR Flip flop




FLIP-FLOPSYMBOL

| S | R | $Q_{(next)}$ |
|---|---|---|
| 0 | 0 | Q |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | ? |

CHARACTERISTIC TABLE

$$Q_{(next)} = S + R'Q$$

$$SR = 0$$

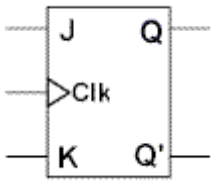
CHARACTERISTIC EQUATION

| Q | $Q_{(next)}$ | S | R |
|---|---|---|---|
| 0 | 0 | 0 | X |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | X | 0 |

EXCITATION TABLE

# FLIPFLOPS:EXCITATION FUNCTIONS

## JK Flip flop




FLIP-FLOPSYMBOL

| J | K | Q(next) |
|---|---|---|
| 0 | 0 | Q |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | Q' |

CHARACTERISTIC TABLE

$$Q_{(next)} = JQ' + K'Q$$

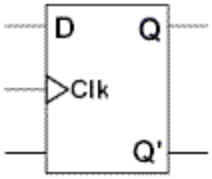
CHARACTERISTIC EQUATION

| Q | Q(next) | J | K |
|---|---|---|---|
| 0 | 0 | 0 | X |
| 0 | 1 | 1 | X |
| 1 | 0 | X | 1 |
| 1 | 1 | X | 0 |

EXCITATION TABLE

# FLIPFLOPS:EXCITATION FUNCTIONS

## D Flip flop




FLIP-FLOPSYMBOL

| D | $Q_{(next)}$ |
|---|---|
| 0 | 0 |
| 1 | 1 |

CHARACTERISTIC TABLE

$$Q_{(next)} = D$$

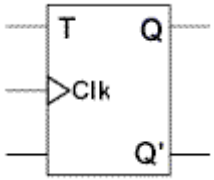
CHARACTERISTIC EQUATION

| Q | $Q_{(next)}$ | D |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

EXCITATION TABLE

# FLIPFLOPS:EXCITATION FUNCTIONS

## T Flip flop




FLIP-FLOPSYMBOL

| T | Q(next) |
|---|---|
| 0 | Q |
| 1 | Q' |

CHARACTERISTIC TABLE

$$Q_{(next)} = TQ' + T'Q$$

CHARACTERISTIC EQUATION

| Q | Q(next) | T |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

EXCITATION TABLE

# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF SR FLIP FLOP TO JK FLIPFLOP

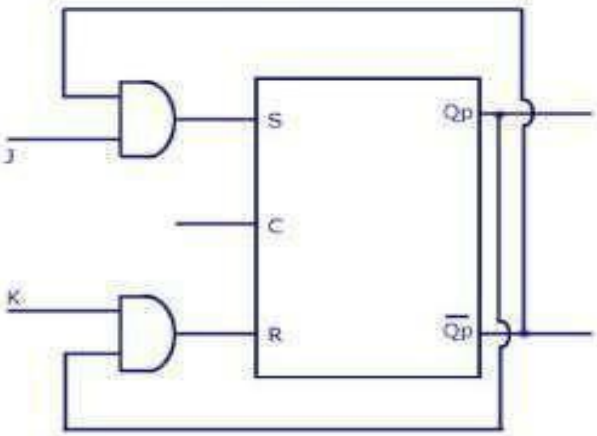
J and K will be given as external inputs to S and R. As shown in the logic diagram below, S and R will be the outputs of the combinational circuit.
The truth tables for the flip flop conversion are given below. The present state is represented by Qp and Qp+1 is the next state to be obtained when the J and K inputs are applied.
For two inputs J and K, there will be eight possible combinations. For each combination of J, K and Qp, the corresponding Qp+1 states are found. Qp+1 simply suggests the future values to be obtained by the JK flip flop after the value of Qp. The table is then completed by writing the values of S and R required to get each Qp+1 from the corresponding Qp. That is, the values of S and R that are required to change the state of the flip flop from Qp to Qp+1 are written.

# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

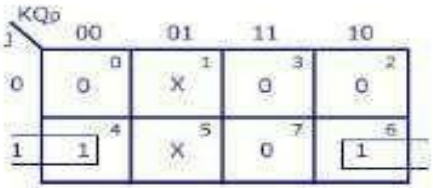
Conversion Table

| J-K Inputs | | Outputs | | S-R Inputs | |
|---|---|---|---|---|---|
| J | K | Qp | Qp+1 | S | R |
| 0 | 0 | 0 | 0 | 0 | X |
| 0 | 0 | 1 | 1 | X | 0 |
| 0 | 1 | 0 | 0 | 0 | X |
| 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 1 | X | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 1 |

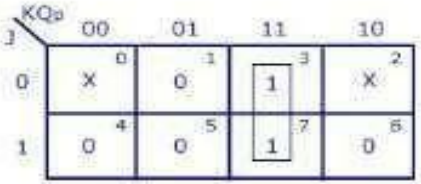
Logic Diagram




SR Flip Flop to JK Flip Flop




$S = \overline{J}Q_p$




$R = KQ_p$

K-Map

www.CircuitsToday.com

SR Flip Flop to JK Flip Flop

# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF JK FLIP FLOP TO SR FLIP FLOP

This will be the reverse process of the above explained conversion. S and R will be the external inputs to J and K. As shown in the logic diagram below, J and K will be the outputs of the combinational circuit. Thus, the values of J and K have to be obtained in terms of S, R and Qp. The logic diagram is shown below.

A conversion table is to be written using S, R, Qp, Qp+1, J and K. For two inputs, S and R, eight combinations are made. For each combination, the corresponding Qp+1 outputs are found ut. The outputs for the combinations of S=1 and R=1 are not permitted for an SR flip flop. Thus the outputs are considered invalid and the J and K values are taken as ―don't cares‖.

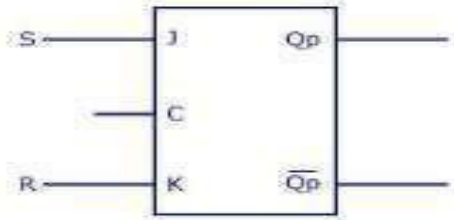
# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

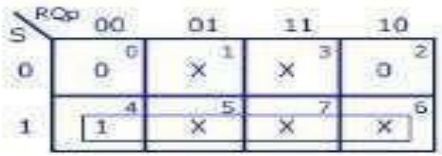
## J-K Flip Flop to S-R Flip Flop

Conversion Table

| S-R Inputs | | Outputs | | J-K Inputs | |
|---|---|---|---|---|---|
| S | R | Qp | Qp+1 | J | K |
| 0 | 0 | 0 | 0 | 0 | X |
| 0 | 0 | 1 | 1 | X | 0 |
| 0 | 1 | 0 | 0 | 0 | X |
| 0 | 1 | 1 | 0 | X | 1 |
| 1 | 0 | 0 | 1 | 1 | X |
| 1 | 0 | 1 | 1 | X | 0 |
| 1 | 1 | Invalid | | Dont care | |
| 1 | 1 | Invalid | | Dont care | |

Logic Diagram




J=S

K-maps

K=R

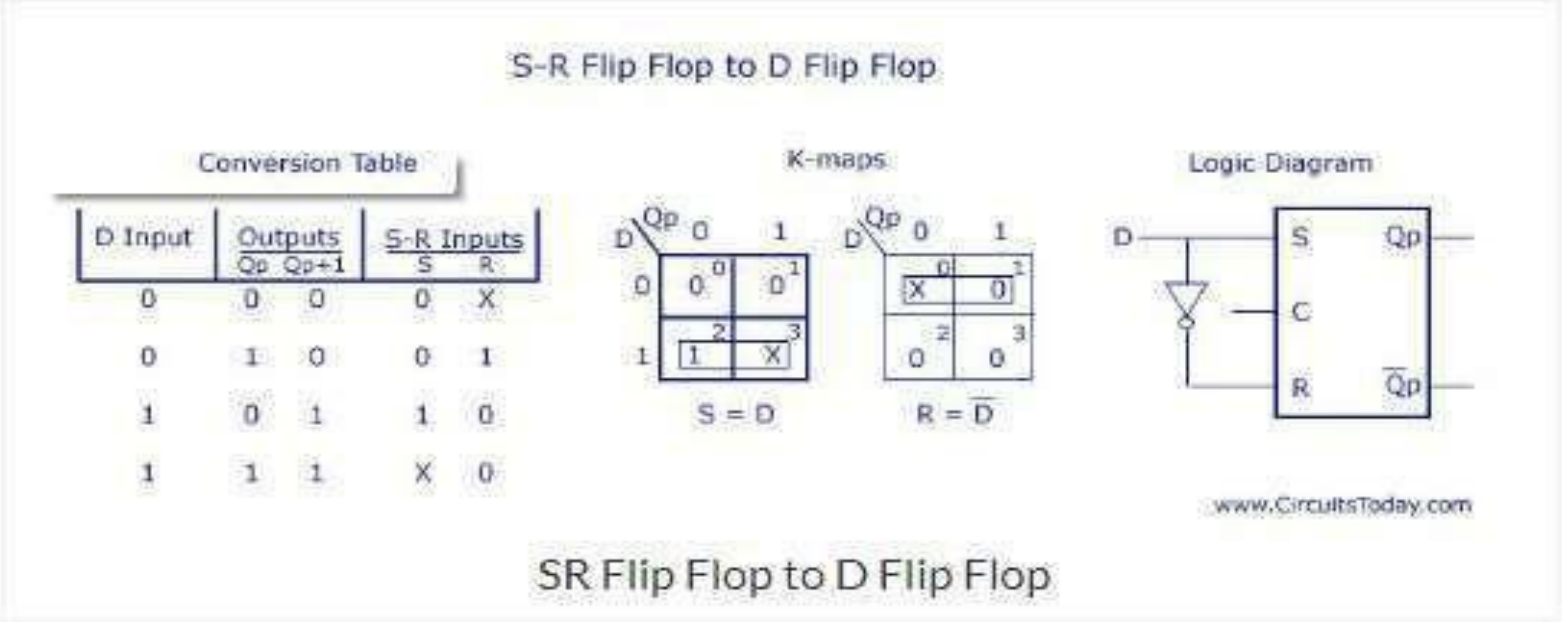
www.CircuitsToday.com

JK Flip Flop to SR Flip Flop

# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF SR FLIP FLOP TO D FLIPFLOP

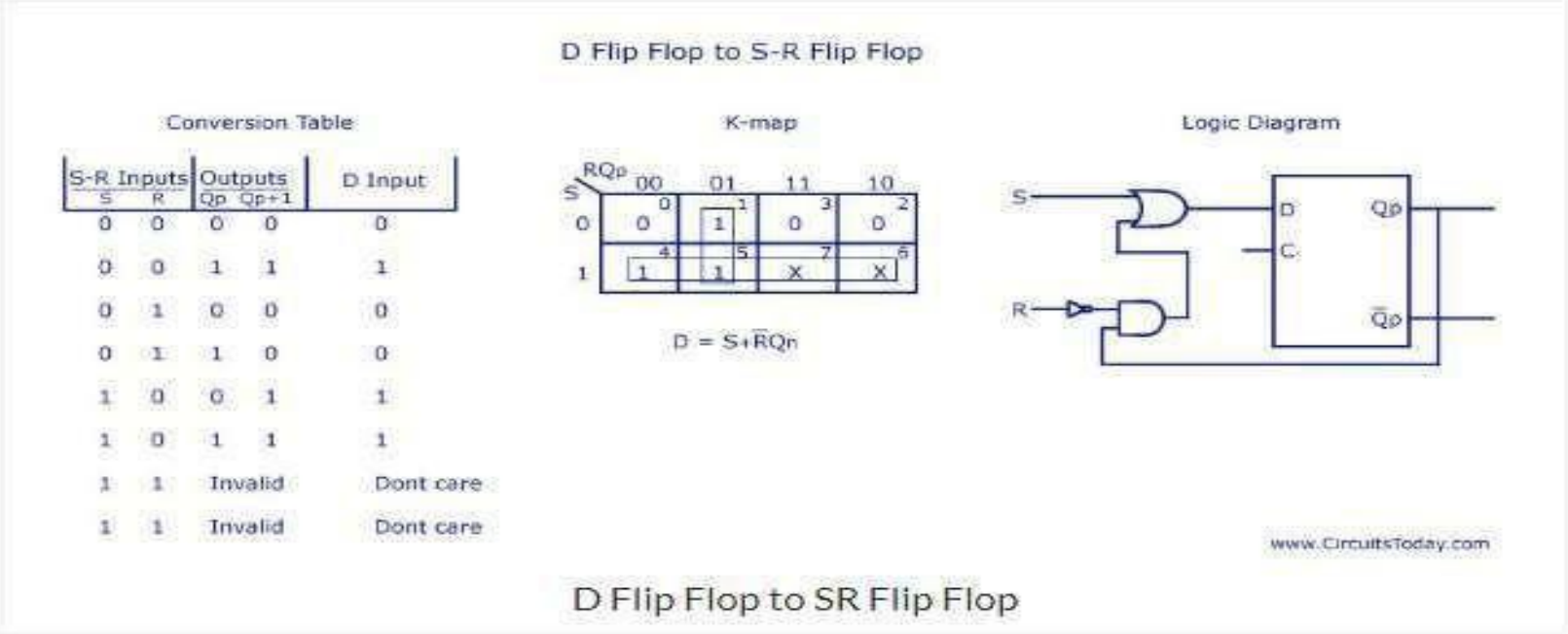
As shown in the figure, S and R are the actual inputs of the flip flop and D is the external input of the flip flop. The four combinations, the logic diagram, conversion table, and the K-map for S and R in terms of D and Qp are shown below.



# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF D FLIP FLOP TO SR FLIPFLOP

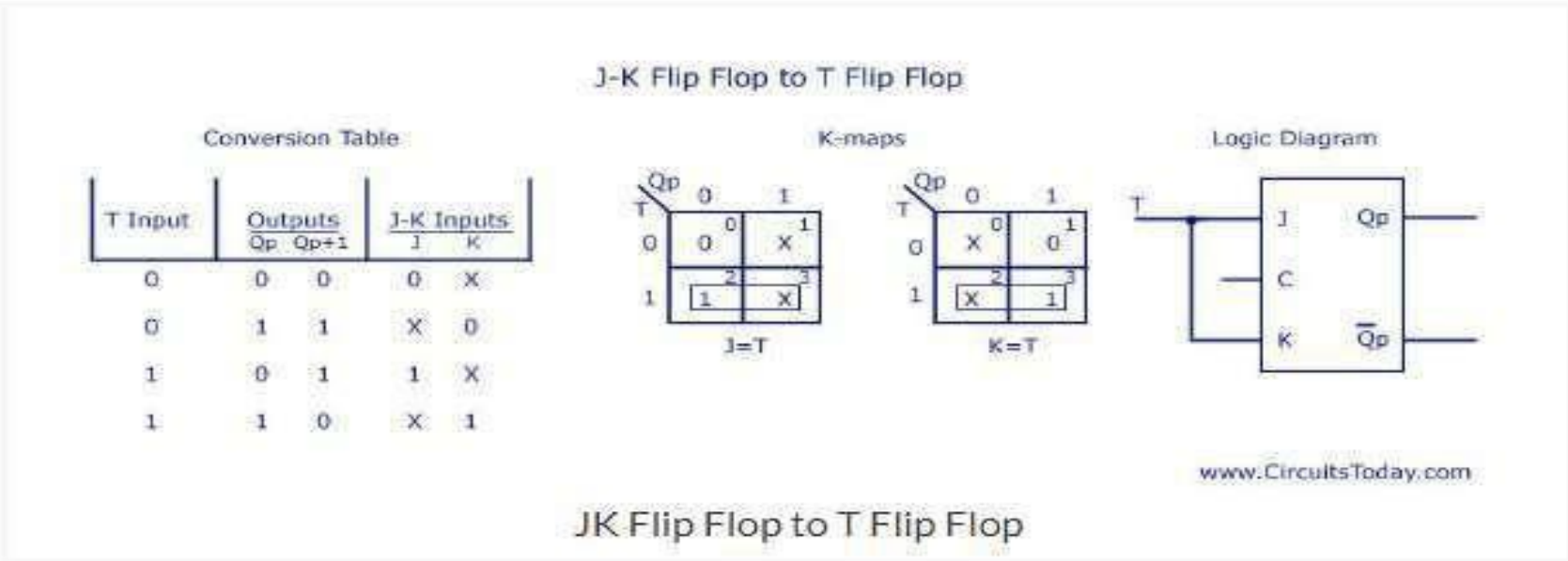
D is the actual input of the flip flop and S and R are the external inputs. Eight possible combinations are achieved from the external inputs S, R and Qp. But, since the combination of S=1 and R=1 are invalid, the values of Qp+1 and D are considered as ―don't cares‖. The logic diagram showing the conversion from D to SR, and the K-map for D in terms of S, R and Qp are shown below.



# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF JK FLIP FLOP TO T FLIP FLOP

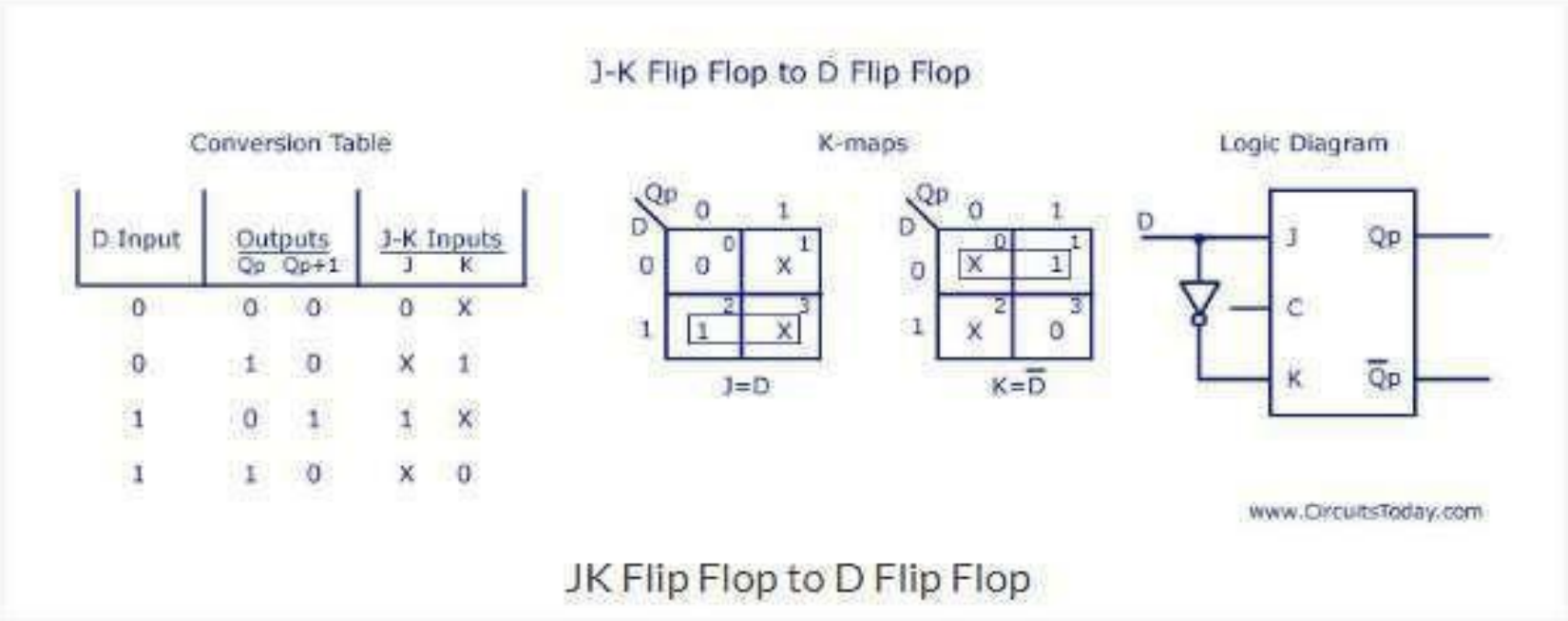
J and K are the actual inputs of the flip flop and T is taken as the external input for conversion. Four combinations are produced with T and Qp. J and K are expressed in terms of T and Qp. The conversion table, K-maps, and the logic diagram are given below.



# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF JK FLIP FLOP TO D FLIPFLOP

D is the external input and J and K are the actual inputs of the flip flop. D and Qp make four combinations. J and K are expressed in terms of D and Qp. The four combination conversion table, the K-maps for J and K in terms of D and Qp, and the logic diagram showing the conversion from JK to D are given below.

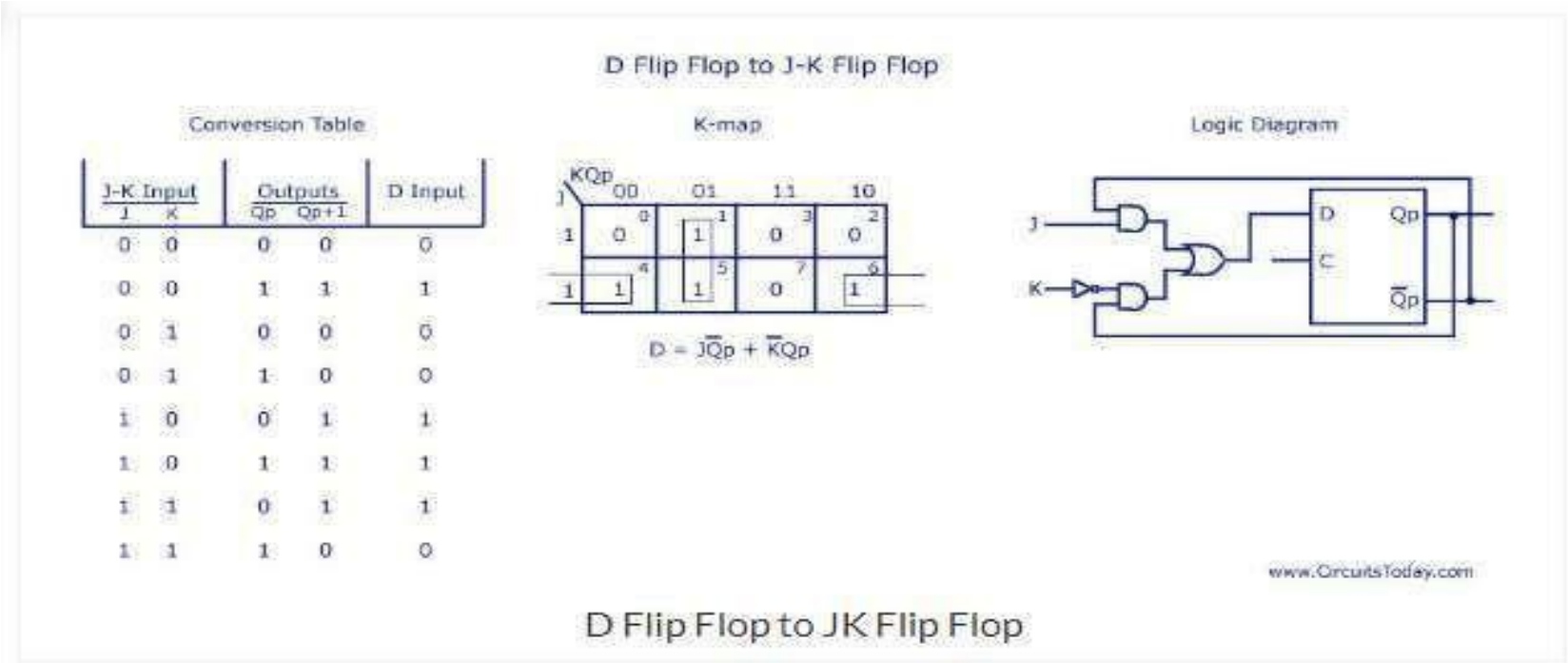


# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF D FLIP FLOP TO JK FLIPFLOP

In this conversion, D is the actual input to the flip flop and J and K are the external inputs. J, K and Qp make eight possible combinations, as shown in the conversion table below. D is expressed in terms of J, K and Qp.

The conversion table, the K-map for D in terms of J, K and Qp and the logic diagram showing the conversion from D to JK are given in the figure below.



# CONVERTION OF ONE FLIP FLOP TO ANOTHER FLIP FLOP

## CONVERTION OF T FLIP FLOP TO JK FLIP FLOP

We begin with the T-to-JK conversion table (see Figure 5), which combines the information in the JK flip-flop's truth table and the T flip-flop's excitation table.

Truth Table of JK Flip-flop

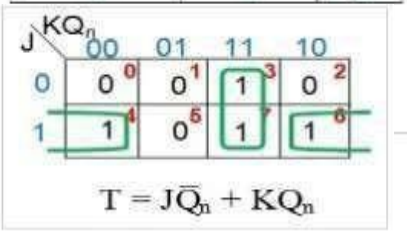
| Inputs | | Outputs | |
|---|---|---|---|
| | | Present State | Next State |
| J | K | $Q_n$ | $Q_{n+1}$ |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

| JK Inputs | | Outputs | | T Input |
|---|---|---|---|---|
| | | Present State | Next State | |
| J | K | $Q_n$ | $Q_{n+1}$ | T |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |

T to JK Conversion Table

Excitation Table of T Flip-flop

| Outputs | | Input |
|---|---|---|
| Present State | Next State | |
| $Q_n$ | $Q_{n+1}$ | T |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |




$T = J\overline{Q}_n + KQ_n$

Next, we need to obtain the simplified Boolean expression for the T input in terms of J, K, and $Q_n$. The expression for the T input as $J\overline{Q}_n + KQ_n$. This means that to convert the T flip-flop into a JK flip-flop, the T input is driven by the output of a two-input OR gate which has as inputs

J ANDed with the negation of the present-state $Q_n$, i.e., $\overline{Q}_n$

K ANDed with the present-state, $Q_n$